

Lists, Attributes, & S3

Lecture 03

Dr. Colin Rundel

Generic Vectors

Lists

Lists are the other vector data structure in R, they differ from atomic vectors in that they are a *heterogeneous* collection of R objects (e.g. atomic vectors, other lists, functions, etc.).

```
1 list("A", c(TRUE,FALSE), (1:4)/2, list(1L), function(x) x^2)
```

```
[[1]]
```

```
[1] "A"
```

```
[[2]]
```

```
[1] TRUE FALSE
```

```
[[3]]
```

```
[1] 0.5 1.0 1.5 2.0
```

```
[[4]]
```

```
[[4]][[1]]
```

```
[1] 1
```

```
[[5]]
```

```
function (x)
```

```
x^2
```

List Structure

Often we want a more compact representation of a complex object; the `str()` function is useful for this, particularly for lists.

```
1 str(c(1,2))
```

```
num [1:2] 1 2
```

```
1 str(1:100)
```

```
int [1:100] 1 2 3 4 5 6 7 8 9 10 ...
```

```
1 str("A")
```

```
chr "A"
```

```
1 str( list(  
2   "A", c(TRUE,FALSE),  
3   (1:4)/2, list(TRUE, 1),  
4   function(x) x^2  
5 ) )
```

List of 5

```
$ : chr "A"
```

```
$ : logi [1:2] TRUE FALSE
```

```
$ : num [1:4] 0.5 1 1.5 2
```

```
$ :List of 2
```

```
..$ : logi TRUE
```

```
..$ : num 1
```

```
$ :function (x)
```

Recursive lists

Lists can contain other lists, meaning they don't have to be flat

```
1 str( list(1, list(2, list(3, 4), 5)) )
```

List of 2

\$: num 1

\$:List of 3

..\$: num 2

..\$:List of 2

.. ..\$: num 3

.. ..\$: num 4

..\$: num 5

Because of this, lists become a natural way of representing *tree-like* structures within R

List Coercion

By default an atomic vector concatenated with a list will be coerced to part of the list (as the list is more generic)

```
1 str( c(1, list(4, list(6, 7))) )
```

List of 3

```
$ : num 1
$ : num 4
$ :List of 2
..$ : num 6
..$ : num 7
```

```
1 str( list(1, list(4, list(6, 7))) )
```

List of 2

```
$ : num 1
$ :List of 2
..$ : num 4
..$ :List of 2
.. ..$ : num 6
.. ..$ : num 7
```

We can coerce a list into an atomic vector using `unlist()` - typical atomic type coercion rules then apply to determine the final type.

```
1 unlist(list(1:3, list(4:5, 6)))
```

```
[1] 1 2 3 4 5 6
```

```
1 unlist( list(1, list(2, list(3, "Hello"))) )
```

```
[1] "1"      "2"      "3"      "Hello"
```

Named lists

Because of their more complex structure we often want to name the elements of a list (we can also do this with atomic vectors).

This can make accessing list elements more straightforward and avoids the use of magic numbers.

```
1 str(list(A = 1, B = list(C = 2, D = 3)))
```

```
List of 2
```

```
$ A: num 1
```

```
$ B:List of 2
```

```
..$ C: num 2
```

```
..$ D: num 3
```

More complex names (i.e. non-valid object names) must be quoted,

```
1 list("knock knock" = "who's there?")
```

```
$`knock knock`
```

```
[1] "who's there?"
```

Aside - Variable names vs. value names

We have seen how to assign a name to an R object (via `=` or `<-`). The general rule for these names is that it must start with a letter (upper or lower) or a `.` and then be followed by additional letters, numbers, `.` or `_`.

These names are unambiguous to the interpreter / parser and so do not need any additional decoration. However if you want to use a name that does not follow these rules, then you must quote it using backticks.

```
1 a b = 1
2 a b
```

```
1 "a b" = 1
2 "a b"
```

```
1 `a b` = 1
2 `a b`
```

```
Error in parse(text = input) [1] "a b"
1: a b
   ^
```

```
[1] 1
```

Vector (atomic or generic) names can be any valid R character vector values (as this is how they are stored) but there are a number of circumstances where we use them like a variable name (e.g. `mtcars$mpg`), and so it is a good idea to avoid using names that violate the object naming rules to avoid having to use backticks all the time (e.g. `x$`knock knock``).

Exercise 1

Represent the following JSON data as a list in R.

```
1 {  
2   "firstName": "John",  
3   "lastName": "Smith",  
4   "age": 25,  
5   "address":  
6   {  
7     "streetAddress": "21 2nd Street",  
8     "city": "New York",  
9     "state": "NY",  
10    "postalCode": 10021  
11  },  
12  "phoneNumber":  
13  [ {  
14    "type": "home",  
15    "number": "212 555-1239"  
16  },  
17    {  
18    "type": "fax",  
19    "number": "646 555-4567"  
20  } ]  
21 }
```

NULL Values

NULLs

NULL is a special value / object within R that represents nothing - it always has length zero and a type and mode of "**NULL**". It also cannot have any attributes.

1 NULL	1 c()	1 double()
NULL	NULL	numeric(0)
1 typeof(NULL)	1 c(NULL)	1 length(double())
[1] "NULL"	NULL	[1] 0
1 mode(NULL)	1 c(1, NULL, 2)	1 typeof(double())
[1] "NULL"	[1] 1 2	[1] "double"
1 length(NULL)	1 c(NULL, TRUE, "A")	1 mode(double())
[1] 0	[1] "TRUE" "A"	[1] "numeric"

0-length coercion

0-length length coercion is a special case of length coercion when one of the arguments has length 0.

In this special case the longer vector will have its length coerced to 0 resulting in a 0-length vector being returned.

```
1 integer() + 1
```

```
numeric(0)
```

```
1 log(numeric())
```

```
numeric(0)
```

```
1 logical() | TRUE
```

```
logical(0)
```

```
1 character() > "M"
```

```
logical(0)
```

As a **NULL** values always have length 0, this rule will apply most of the time (note the types)

```
1 NULL + 1
```

```
numeric(0)
```

```
1 NULL | TRUE
```

```
logical(0)
```

```
1 NULL > "M"
```

```
logical(0)
```

```
1 log(NULL)
```

```
Error in `log()` :  
! non-numeric argument to mathematical function
```

NULLs and comparison

Given the previous issue, comparisons and conditionals with **NULLs** can be problematic.

```
1 x = NULL
```

```
1 if (x > 0)
2   print("Hello")
```

Error in `if (x > 0) ...`:
! argument is of length zero

```
1 if (!is.null(x) & (x > 0))
2   print("Hello")
```

Error in `if (!is.null(x) & (x > 0)) ...`:
! argument is of length zero

```
1 if (!is.null(x) && (x > 0))
2   print("Hello")
```

The null coalescing operator (%||%)

The %||% operator is a useful infix operator that returns the left hand side if it is not **NULL** and the right hand side otherwise.

```
1 ` %||% `
```

```
function (x, y)
if (is.null(x)) y else x
<bytecode: 0x127303990>
<environment: namespace:base>
```

```
1 if (x %||% 0 > 0)
2   print("Hello")
```

```
1 if (x %||% 1 > 0)
2   print("Hello")
```

```
[1] "Hello"
```

This operator is a relatively recent addition to base R (added in v4.4) but was popularized in the tidyverse and other packages.

Attributes

Attributes

Attributes are metadata that can be attached to objects in R.

Certain attributes are special (e.g. `class`, `comment`, `dim`, `dimnames`, `names`, ...) because they change the behavior of the object(s).

Attributes are implemented as a **named list** that is attached to the object. They can be interacted with via the `attr()` and `attributes()` functions.

```
1 (x = c(L=1, M=2, N=3))
```

```
L M N  
1 2 3
```

```
1 str(attributes(x))
```

```
List of 1  
 $ names: chr [1:3] "L" "M" "N"
```

```
1 attr(x, "names")
```

```
[1] "L" "M" "N"
```

```
1 attr(x, "other")
```

```
NULL
```


Assigning attributes

The most commonly used / important attributes will usually have helper functions for getting and setting,

```
1 x
```

```
L M N  
1 2 3
```

```
1 names(x)
```

```
[1] "L" "M" "N"
```

```
1 names(x) = c("Z","Y","X")  
2 x
```

```
Z Y X  
1 2 3
```

```
1 names(x)
```

```
[1] "Z" "Y" "X"
```

```
1 attr(x, "names") = c("A","B","C")  
2 x
```

```
A B C  
1 2 3
```

```
1 names(x)
```

```
[1] "A" "B" "C"
```

Helpers functions vs attr

```
1 names(x) = 1:3
2 x
```

```
1 2 3
```

```
1 2 3
```

```
1 attributes(x)
```

```
$names
```

```
[1] "1" "2" "3"
```

```
1 attr(x, "names") = 1:3
2 x
```

```
1 2 3
```

```
1 2 3
```

```
1 attributes(x)
```

```
$names
```

```
[1] "1" "2" "3"
```

```
1 names(x) = c(TRUE, FALSE, TRUE)
2 x
```

```
TRUE FALSE TRUE
```

```
1 2 3
```

```
1 attributes(x)
```

```
$names
```

```
[1] "TRUE" "FALSE" "TRUE"
```

Note that there are some guardrails in place to ensure that special attributes make sense (e.g. `names` must be a character

Factors

Factor objects are how R represents categorical data (e.g. a variable where there is a discrete set of possible outcomes).

```
1 (x = factor(c("Sunny", "Cloudy", "Rainy", "Cloudy", "Cloudy")))
```

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

```
1 str(x)
```

```
Factor w/ 3 levels "Cloudy","Rainy",...: 3 1 2 1 1
```

```
1 typeof(x)
```

```
[1] "integer"
```

```
1 mode(x)
```

```
[1] "numeric"
```

```
1 class(x)
```

```
[1] "factor"
```

Composition

A factor is just an integer vector with two attributes: `class` and `levels`.

```
1 x
```

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

```
1 str(attributes(x))
```

List of 2

```
$ levels: chr [1:3] "Cloudy" "Rainy" "Sunny"  
$ class : chr "factor"
```

We can build our own factor from scratch using `attr()`,

```
1 y = c(3L, 1L, 2L, 1L, 1L)  
2 attr(y, "levels") = c("Cloudy", "Rainy", "Sunny")  
3 attr(y, "class") = "factor"  
4 y
```

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

Building objects

The approach we just used is a bit clunky - generally the preferred method for constructing an object with attributes from scratch is to use the `structure()` function.

```
1 ( y = structure(  
2   c(3L, 1L, 2L, 1L, 1L),  
3   levels = c("Cloudy", "Rainy", "Sunny"),  
4   class = "factor"  
5 ) )
```

```
[1] Sunny Cloudy Rainy Cloudy Cloudy  
Levels: Cloudy Rainy Sunny
```

```
1 class(y)
```

```
[1] "factor"
```

```
1 is.factor(y)
```

```
[1] TRUE
```

Factors are integer vectors?

Knowing factors are stored as integers help explain some of their more interesting behaviors:

```
1 x+1
```

```
Warning in Ops.factor(x, 1): '+' not meaningful for factors
```

```
[1] NA NA NA NA NA
```

```
1 is.integer(x)
```

```
[1] FALSE
```

```
1 as.integer(x)
```

```
[1] 3 1 2 1 1
```

```
1 as.character(x)
```

```
[1] "Sunny" "Cloudy" "Rainy" "Cloudy" "Cloudy"
```

```
1 as.logical(x)
```

```
[1] NA NA NA NA NA
```

S3 Object System

class

The `class` attribute is an additional layer to R's type hierarchy,

value	<code>typeof()</code>	<code>mode()</code>	<code>class()</code>
<code>TRUE</code>	logical	logical	logical
<code>1</code>	double	numeric	numeric
<code>1L</code>	integer	numeric	integer
<code>"A"</code>	character	character	character
<code>NULL</code>	NULL	NULL	NULL
<code>list(1, "A")</code>	list	list	list
<code>factor("A")</code>	integer	numeric	factor
<code>function(x) x^2</code>	closure	function	function
<code>+</code>	builtin	function	function
<code>[</code>	special	function	function

S3 class specialization

```
1 x = c("A","B","A","C")
```

```
1 print( x )
```

```
[1] "A" "B" "A" "C"
```

```
1 print( factor(x) )
```

```
[1] A B A C  
Levels: A B C
```

```
1 print( unclass( factor(x) ) )
```

```
[1] 1 2 1 3  
attr(,"levels")  
[1] "A" "B" "C"
```

```
1 print.default( factor(x) )
```

```
[1] 1 2 1 3
```

What's up with print?

```
1 print
```

```
function (x, ...)  
UseMethod("print")  
<bytecode: 0x121d0e550>  
<environment: namespace:base>
```

```
1 print.default
```

```
function (x, digits = NULL, quote = TRUE, na.print = NULL, print.gap = NULL,  
  right = FALSE, max = NULL, width = NULL, useSource = TRUE,  
  ...)  
{  
  args <- pairlist(digits = digits, quote = quote, na.print = na.print,  
    print.gap = print.gap, right = right, max = max, width = width,  
    useSource = useSource, ...)  
  missings <- c(missing(digits), missing(quote), missing(na.print),  
    missing(print.gap), missing(right), missing(max), missing(width),  
    missing(useSource))  
  .Internal(print.default(x, args, missings))  
}  
<bytecode: 0x110be5c18>  
<environment: namespace:base>
```

Other examples

1 mean

```
function (x, ...)
UseMethod("mean")
<bytecode: 0x1222eac50>
<environment: namespace:base>
```

1 summary

```
function (object, ...)
UseMethod("summary")
<bytecode: 0x110cf7748>
<environment: namespace:base>
```

1 t.test

```
function (x, ...)
UseMethod("t.test")
<bytecode: 0x120476900>
<environment: namespace:stats>
```

1 plot

```
function (x, y, ...)
UseMethod("plot")
<bytecode: 0x110d31c18>
<environment: namespace:base>
```

A lot but not all base functions use this approach,

1 sum

```
function (... , na.rm = FALSE) .Primitive("sum")
```

What is S3?

S3 is R's first and simplest OO system. It is the only OO system used in the base and stats packages, and it's the most commonly used system in CRAN packages. S3 is informal and ad hoc, but it has a certain elegance in its minimalism: you can't take away any part of it and still have a useful OO system.

— Hadley Wickham, Advanced R

What's going on?

S3 objects and their related functions work using a very simple dispatch mechanism - a generic function is created whose sole job is to call the `UseMethod` function which then calls a class specialized function using the naming convention: `<generic>.`

`<class>`

We can see all of the specialized versions of the generic using the `methods` function.

```
1 methods("plot")
```

```
[1] plot.acf*           plot.data.frame*    plot.decomposed.ts*
[4] plot.default        plot.dendrogram*    plot.density*
[7] plot.ecdf           plot.factor*         plot.formula*
[10] plot.function       plot.hclust*         plot.histogram*
[13] plot.HoltWinters*    plot.isoreg*         plot.lm*
[16] plot.medpolish*     plot.mlm*            plot.ppr*
[19] plot.prcomp*        plot.princomp*       plot.profile*
[22] plot.profile.nls*   plot.raster*         plot.spec*
[25] plot.stepfun        plot.stl*            plot.table*
[28] plot.ts             plot.tskernel*       plot.TukeyHSD*
see '?methods' for accessing help and source code
```

Other examples

1 `methods("print")`

```
[1] print.acf*
[2] print.activeConcordance*
[3] print.AES*
[4] print.anova*
[5] print.aov*
[6] print.aovlist*
[7] print.ar*
[8] print.Arima*
[9] print.arima0*
[10] print.AsIs
[11] print.aspell*
[12] print.aspell_inspect_context*
[13] print.bibentry*
[14] print.Bibtex*
[15] print.browseVignettes*
[16] print.by
[17] print.changedFiles*
[18] print.check_bogus_return*
[19] print.check_code_usage_in_package*
[20] print.check_compiled_code*
```

1 `print.factor`

```
function (x, quote = FALSE, max.levels = N
  ...)
{
  ord <- is.ordered(x)
  if (length(x) == 0L)
    cat(if (ord)
        "ordered"
        else "factor", "()\n", sep = "")
  else {
    xx <- character(length(x))
    xx[] <- as.character(x)
    keepAttrs <- setdiff(names(attributes(
      "class")))
    attributes(xx)[keepAttrs] <- attri
    print(xx, quote = quote, ...)
  }
  maxl <- max.levels %||% TRUE
  if (maxl) {
    n <- length(lev <- encodeString(lev
      "\"", "")))
```

The other way

If instead we have a class and want to know what specialized functions exist for that class, then we can again use the `methods` function with the `class` argument.

```
1 methods(class="factor")
```

```
[1] [          [[          [[<-          [<-          all.equal
[6] as.character as.data.frame as.Date       as.list       as.logical
[11] as.POSIXlt   as.vector      c             coerce        droplevels
[16] format       initialize     is.na<-      length<-     levels<-
[21] Math         Ops           plot         print         relevel
[26] relist       rep          show         slotsFromS3   summary
[31] Summary      xtfrm

see '?methods' for accessing help and source code
```

Adding methods

```
1 ( x = structure(  
2   c(1,2,3),  
3   class="class_A") )
```

```
[1] 1 2 3  
attr(,"class")  
[1] "class_A"
```

```
1 print.class_A = function(x) {  
2   cat("(Class A) ")  
3   print.default(unclass(x))  
4 }  
5 print(x)
```

```
(Class A) [1] 1 2 3
```

```
1 class(x) = "class_B"  
2 print(x)
```

```
(Class B) [1] 1 2 3
```

```
1 ( y = structure(  
2   c(6,5,4),  
3   class="class_B") )
```

```
[1] 6 5 4  
attr(,"class")  
[1] "class_B"
```

```
1 print.class_B = function(x) {  
2   cat("(Class B) ")  
3   print.default(unclass(x))  
4 }  
5 print(y)
```

```
(Class B) [1] 6 5 4
```

```
1 class(y) = "class_A"  
2 print(y)
```

```
(Class A) [1] 6 5 4
```


Defining a new S3 Generic

```
1 shuffle = function(x) {  
2   UseMethod("shuffle")  
3 }
```

```
1 shuffle.default = function(x) {  
2   stop("Class ", class(x), " is not supported by shuffle.", call. =  
3 }
```

```
1 shuffle.factor = function(f) {  
2   factor( sample(as.character(f)), levels = sample(levels(f)) )  
3 }
```

```
1 shuffle.integer = function(x) {  
2   sample(x)  
3 }
```

Shuffle results

```
1 shuffle( 1:10 )
```

```
[1] 1 9 7 8 3 2 4 5 10 6
```

```
1 shuffle( factor(c("A","B","C","A")) )
```

```
[1] A B C A
```

```
Levels: C A B
```

```
1 shuffle( c(1, 2, 3, 4, 5) )
```

Error:

! Class numeric is not supported by shuffle.

```
1 shuffle( letters[1:5] )
```

Error:

! Class character is not supported by shuffle.

```
1 shuffle( factor(letters[1:5]) )
```

```
[1] e d c b a
```

```
Levels: e b a d c
```

Demo - classes, modes, and types

```
1 report = function(x) {  
2   UseMethod("report")  
3 }  
4 report.default = function(x) {  
5   paste0("Class ", class(x), " does not have a method  
6 }  
7 report.integer = function(x) {  
8   "I'm an integer!"  
9 }  
10 report.double = function(x) {  
11   "I'm a double!"  
12 }  
13 report.numeric = function(x) {  
14   "I'm a numeric!"  
15 }  
16  
17 #rm(report.integer)  
18 #rm(report.double)  
19 #rm(report.numeric)  
20  
21 report(1)  
22 report(1L)  
23 report("1")
```

On the left we have defined an S3 method called `report`, it is designed to return a message about the type/mode/class of an object passed to it.

- Try running the `report` function with different input types, what happens?
- Now uncomment out the first `rm()` line and try rerunning the code, what has changed?
- What does this tell us about S3, types, modes, and classes?
- What if we also uncomment the code the other `rm()` lines?

Conclusions?

From [UseMethods](#) R documentation:

If the object does not have a class attribute, it has an implicit class. Matrices and arrays have class “matrix” or “array” followed by the class of the underlying vector. Most vectors have class the result of `mode(x)`, except that integer vectors have class `c("integer", "numeric")` and real vectors have class `c("double", "numeric")`.

From Advanced R:

How does `UseMethod()` work? It basically creates a vector of method names, `paste0("generic", ".", c(class(x), "default"))`, and then looks for each potential method in turn.

Why?



"For compatibility with S"